

- > Main drawback of the perceptron trick is, we will get a line from it, but we can't say the line is perfect for this data. Cause we are not using any loss function here to say that the model is perfect cause the loss function is minimum here, we won't get more.
- > Perceptron don't use any loss function, that's why it is not optimal.
- > Since it only do action when it find that any point is missclassified, when it divide all of the data into classes it won't do anything farther to find the optimal solution.
- > Loss function tells us, how well the model is, or we can say how much error present in this model.
- > So, the goal is to minimize that function by changing the values of weights and biases.
- > To do this we use gradient descent.

The screenshot shows a digital drawing application interface with a dark background. At the top, there is a menu bar with options: D5 Perceptron Loss Functions, HOME, INSERT, DRAW, and VIEW. Below the menu bar is a toolbar with various drawing tools. The main drawing area contains handwritten mathematical formulas in white ink. The first formula is the total loss function:
$$L(w_1, w_2, b) = \frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i)) + \alpha R(w_1, w_2)$$
 An arrow points from the text "Regularization" to the term $\alpha R(w_1, w_2)$. The second formula is the individual loss function:
$$L(y_i, f(x_i)) = \max(0, -y_i f(x_i))$$
 An arrow points from the term $f(x_i)$ to its definition:
$$f(x_i) = w_1 x_1 + w_2 x_2 + b$$
 Below this definition, the text "Z =" is written.

7:29 68%

D5 Perceptron Loss Functions HOME INSERT DRAW VIEW

Stop

$$L(y_i, f(x_i)) = \max(0, -y_i f(x_i))$$

$$f(x_i) = w_1 x_1 + w_2 x_2 + b$$

$z =$

$$L = \frac{1}{n} \sum_{i=1}^n \max(0, -y_i f(x_i))$$

n : # rows in data

$L(\underline{w}, \underline{w}_2, \underline{b})$

$L = \underset{\underline{w}, \underline{w}_2, \underline{b}}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^n \max(0, -y_i f(x_i))$ minimum

-> It is the perceptron Loss Function, and this loss function is used by stlearn

-> At the bottom pdf, it shows how the loss function works here.

-> For correctly classified points, the $\max(0, -x)$ becomes 0. cause actual and predicted point will be same (either possitive or negative each) $\max(0, -x)$ becomes 0 cause negative is less than 0

-> After all the correctly classified points won't do any effect.

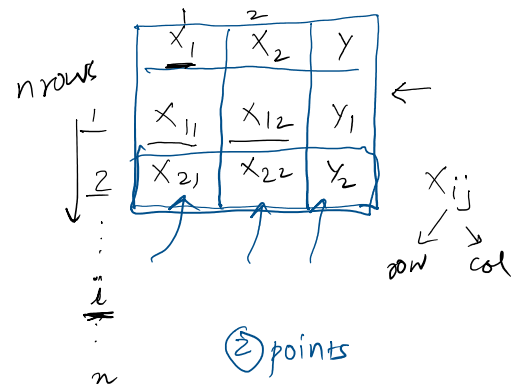
-> miss classified points will leave some numbers and we sum it and avg it, we will get a error. and we are trying to miniize is by finding the perfect line that classify maximum points correctly.

Explanation of Loss Function

23 February 2022 08:07

$$L = \frac{1}{n} \sum_{i=1}^n \max(0, -y_i f(x_i))$$

where $f(x_i) = w_1 x_{i1} + w_2 x_{i2} + b$



$$\max(0, -y_i f(x_i)) = \max(0, x) \rightarrow \begin{cases} x \geq 0 \\ x < 0 \end{cases}$$

where $x = -y_i f(x_i)$

$$f(x_2) = w_1 x_{21} + w_2 x_{22} + b$$

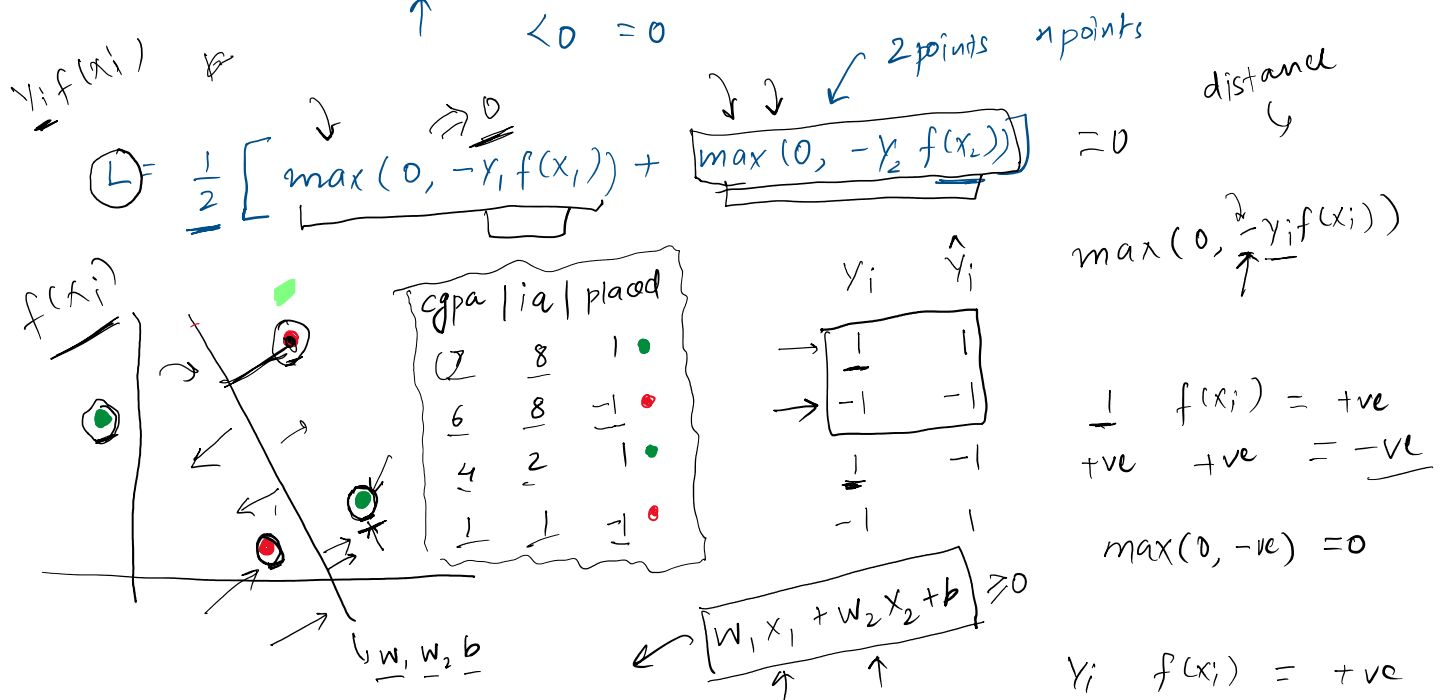


Diagram illustrating the loss function L as a sum of $\max(0, -y_i f(x_i))$ terms. The diagram shows a 2D input space with a decision boundary. Points are labeled $x_1, x_2, \dots, x_n$ and the output is y . A point x_{ij} is highlighted, showing its row i and column j .

Diagram illustrating the loss function L as a sum of $\max(0, -y_i f(x_i))$ terms. The diagram shows a 2D input space with a decision boundary. Points are labeled $x_1, x_2, \dots, x_n$ and the output is y . A point x_{ij} is highlighted, showing its row i and column j .

Gradient Descent

23 February 2022 08:50

$$L = \underset{w_1, w_2, b}{\operatorname{argmin}} \left[\frac{1}{n} \sum_{j=1}^n \max(0, -y_j f(x_j)) \right]$$

$$L(w_1, w_2, b)$$

$$L(w_1)$$



where $f(x_i) = w_1 x_{i1} + w_2 x_{i2} + b$
 100/1000

$$\eta = 0.01$$

$$\left[\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \frac{\partial L}{\partial b} \right]$$

for j in epochs:

$$w_1 = \underline{w_1} + \eta \frac{\partial L}{\partial w_1}$$

$$w_2 = \underline{w_2} + \eta \frac{\partial L}{\partial w_2}$$

$$b = \underline{b} + \eta \frac{\partial L}{\partial b}$$

-> Gradient Descent to Minimize the Loss Function

Loss Function Differentiation

23 February 2022 12:57

$$L = \frac{1}{n} \sum_{i=1}^n \max(0, -y_i f(x_i))$$

where $f(x_i) = w_1 x_{i1} + w_2 x_{i2} + b$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial f(x_i)} \times \frac{\partial f(x_i)}{\partial w_1}$$

$$\frac{\partial L}{\partial f(x_i)} = \begin{cases} 0 & \text{if } y_i f(x_i) \geq 0 \\ -y_i & \text{if } y_i f(x_i) < 0 \end{cases}$$

$$\frac{\partial f(x_i)}{\partial w_1} = x_{i1}$$

$$\frac{\partial L}{\partial w_1} = \begin{cases} 0 & \text{if } y_i f(x_i) \geq 0 \\ -y_i x_{i1} & \text{if } y_i f(x_i) < 0 \end{cases}$$

$$\frac{\partial L}{\partial w_2} = \begin{cases} 0 & \text{if } y_i f(x_i) \geq 0 \\ -y_i x_{i2} & \text{if } y_i f(x_i) < 0 \end{cases}$$

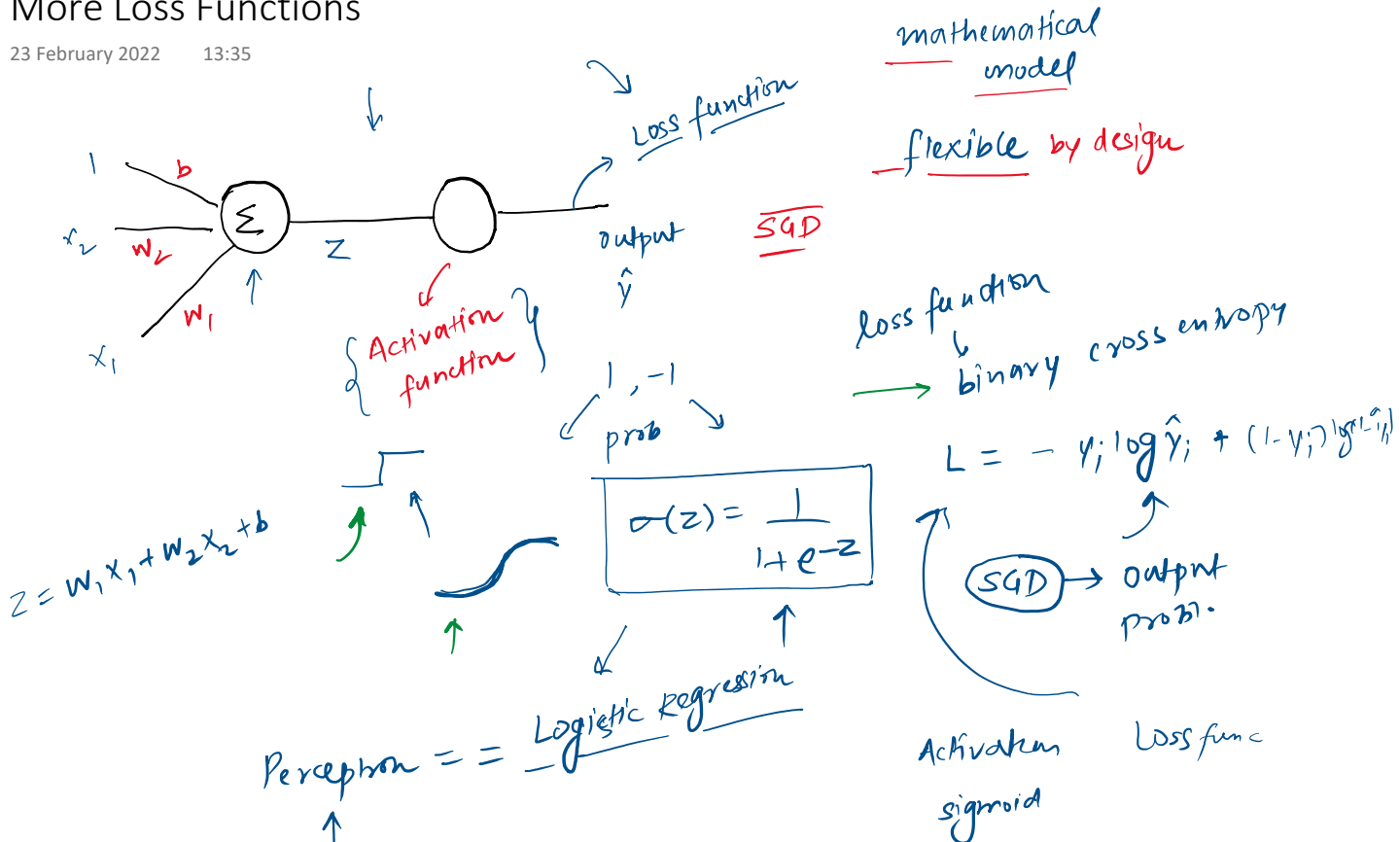
$$\frac{\partial L}{\partial b} = \begin{cases} 0 & \text{if } y_i f(x_i) \geq 0 \\ -y_i & \text{if } y_i f(x_i) < 0 \end{cases}$$

→ code using gradient descent

-> Differentiation of the Loss Function

More Loss Functions

23 February 2022 13:35



multiclass ③

Activation

softmax

$$f = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

$$f = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}}$$

Regression

Activation → Linear



Loss function

categorical cross entropy

$$L = \sum_{j=1}^M y_j \log(\hat{f}_j)$$

Softmax

regression

$$\begin{aligned} z \geq 0 & \rightarrow 1 \\ z < 0 & \rightarrow -1 \end{aligned}$$

$$\hat{z} \rightarrow \hat{z}$$

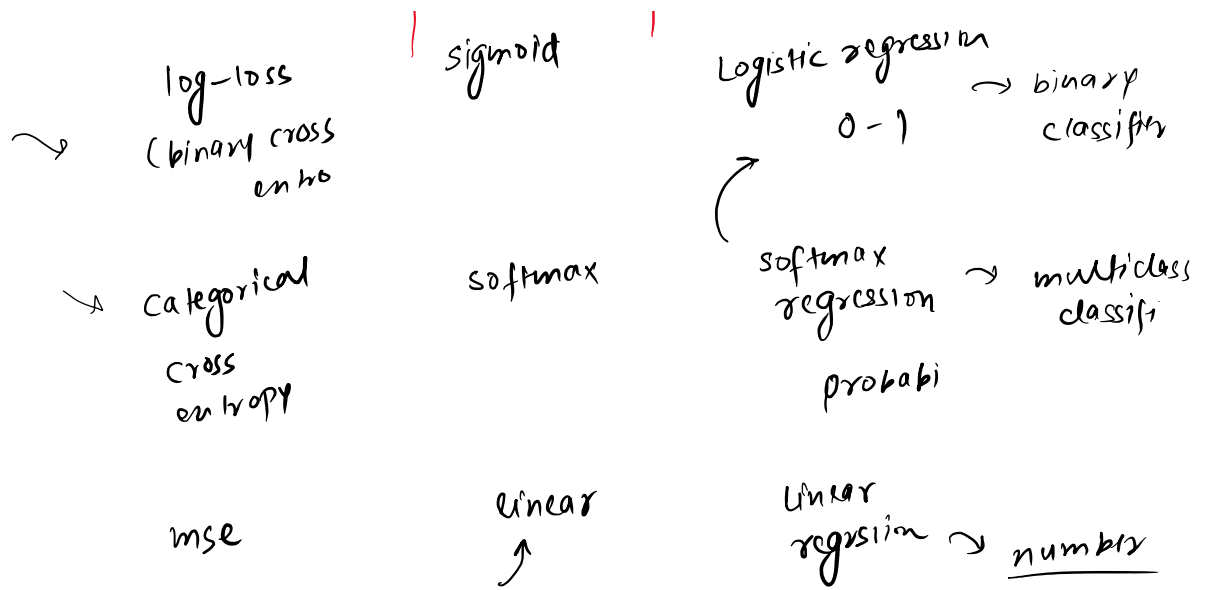
Loss function

mse

Linear regression

$$(y_i - \hat{y}_i)^2$$

Loss function	Activation	output
Hinge loss	step	perceptron → binary classification -1 1
log-loss	sigmoid	Logistic regression → binary



-> Perceptron is a mathematical model that can be used in multiple ways

-> e.g. If we use sigmoid function with cross entropy loss function it becomes logistic regression

-> If we use linear activation function with mean square error loss function, it becomes linear regression.

